FAST — NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES
FAST School of Computing · Karachi Campus · Department of Artificial Intelligence & Data Science
AL-2002 Artificial Intelligence (Laboratory) · Fall 2026

PROJECT PROPOSAL

Semester Lab Project · Weightage 1 mark

Group Information

	Name	Roll Number	Section	Role in the project
Leader	Zumair Shamsi	24K-2040	BCY-5A	Optimization algorithms, A* Search implementation, Telegram dispatcher
Member 2	Huzaifa Zafar	24K-2044	BCY-5A	Data pipeline, feature engineering, Scikit-Learn ML regression, offline evaluation

Project Title	Autonomous Fantasy Premier League Decision-Support Agent
Project Idea	Idea No. _____ from the Project Ideas list ☐ Own idea (needs approval) ✓
Category	Part A — Game ☐ Part B — Applied / Data ☐
GitHub Repository URL	https://github.com/Zumairshamsi/Autonomous-FPL-Decision-Support-Agent
Date of Submission	21/10/2026

1. Project Title and Summary

Autonomous Fantasy Premier League Decision-Support Agent

This software acts as an automated, non-intrusive decision-support assistant for Fantasy Premier League (FPL) managers. A user configures their public FPL Team ID once, and the system continuously tracks squad valuations, fixture schedules, injury statuses, and player statistics. Prior to weekly deadlines, it predicts expected points, executes an informed state-space search to select the optimal starting eleven, captain, and multi-week transfer moves, and dispatches actionable instructions directly to the manager via Telegram. Standard programs cannot solve this problem because greedy rules cannot navigate multi-week transfer cascades, strict £100.0m budget caps, and non-stationary player form. Artificial intelligence makes this possible by combining supervised expected-point prediction with custom informed search (A* search with an admissible pruning heuristic) to discover optimal squad management policies without violating game rules.

2. Problem Statement and Motivation

Selecting an optimal Fantasy Premier League squad across 38 game weeks is a computationally intractable, multi-period combinatorial optimization problem under stochastic real-world outcomes. From a pool of over 650 active Premier League footballers, an agent must select 15 players within a rigid £100.0m budget while adhering to exact positional allotments and a maximum limit of 3 players per club. In any single game week, picking 11 starters from 15 players yields 1,365 valid line-up combinations, which balloons exponentially into millions of states when planning multi-week transfer cascades, bench hierarchies, and point-hit deductions (-4 points per extra transfer). Over

11 million global participants face this challenge weekly, mostly relying on anecdotal punditry, gut feeling, and cognitive pitfalls like recency bias and emotional attachment to underperforming players. Solving this with AI is worthwhile because it replaces human bias with principled probabilistic forecasting and optimal state-space search, showing how utility-driven agents can make superior decisions in complex, budget-constrained stochastic environments.

3. Problem Formulation

3.1 PEAS Description

Component	Description for your project
Performance measure	Cumulative net fantasy points across the season, net point gain over the naive greedy baseline, prediction Root Mean Squared Error (RMSE), and captain pick success rate (% finishing in top 3 team scorers).
Environment	The Premier League match ecosystem: 20 clubs, 38 gameweeks, 650+ players, fluctuating valuations, dynamic injury flags, and home/away fixture difficulties.
Actuators	Formatted Telegram alert dispatcher (outputting starting XI, captain/vice-captain, transfers, and chip alerts) and local JSON/SQLite state store.
Sensors	REST client polling the official Premier League JSON endpoints (bootstrap-static, element-summary, fixtures, and picks).

3.2 Environment and Agent Type

Property	Classification — tick one
Observability	Fully observable ☐ Partially observable ✓
Determinism	Deterministic ☐ Stochastic ✓
Change over time	Static ☐ Dynamic ✓
State and actions	Discrete ✓ Continuous ☐
Agents	Single-agent ✓ Multi-agent / adversarial ☐
Agent type	Simple reflex ☐ Model-based ☐ Goal-based ☐ Utility-based ✓ Learning ☐

3.3 Problem Representation

Element	Your representation
State representation	A structured state tuple: (squad_ids, bank_balance, free_transfers, available_chips, gameweek_index), where squad_ids is an array of 15 player identifiers conforming to positional bounds.
Actions / moves	Discrete actions: Transfer(player_out, player_in), SetStartingXI(11_player_subset), AssignCaptain(player_id), or RollTransfer().
Initial state	The manager's public 15-man squad extracted from the FPL API at gameweek t_0, remaining bank balance, and accumulated free transfers (1 to 5).
Goal or terminal test	Terminal state reached when Gameweek 38 concludes, or locally when a fully valid, constraint-satisfying lineup is generated for the imminent gameweek deadline.
Cost, utility, fitness or evaluation	$U = \sum_{t=1}^{H} \gamma^{t-1} \cdot \left(\sum_{i \in \mathrm{XI}} x\, P_{i,t} + x P_{\mathrm{captain},t} \right) - 4 \cdot \max(0, \mathrm{TransfersUsed} - \mathrm{FreeTransfers})$ where $\gamma = 0.85$ is a temporal discount factor penalizing uncertainty in distant gameweeks.

4. Proposed AI Approach

#	Technique	Role in the project	Own code or library
1	Informed A* Search / Branch-and-Bound	Explores the combinatorial transfer state-space over a 3-week lookahead horizon to find the optimal squad.	**Own code** *(no black-box search library)*
2	Supervised Regressor (Random Forest / Gradient Boosting)	Predicts expected points (xP) per player based on rolling form, minutes played, and fixture difficulty.	Library (scikit-learn)
3	Local Beam Search	Evaluates alternative multi-gameweek transfer candidates when the full A* frontier exceeds memory limits.	Own code
B	Baseline: Greedy Form Heuristic Agent	Naively transfers in the highest past point-scorers within budget with no multi-week lookahead and a static 3-4-3 formation.	Own code

FPL cannot be solved by pure search due to exponential branching across multiple game weeks, nor by pure machine learning because regression models cannot enforce knapsack and formation constraints. Hence, this hybrid approach is optimal. First, a Scikit-Learn regressor predicts continuous expected points (xP) from historical metrics (rolling xG, xA, minutes, and fixture ratings).
Second, our custom A* search algorithm uses these predictions to explore transfers over a rolling lookahead horizon. The heuristic function $h(n)$ estimates the maximum possible points achievable by an ideal, unconstrained squad over the remaining horizon. Because no constrained squad can score more than the unconstrained optimum, $h(n)$ is strictly admissible ($h(n) \geq h^*(n)$ for maximization), guaranteeing an optimal lineup search while pruning low-scoring combinations.

4.1 Planned Class Design

1. **FPLEnvironment:**
   - *Attributes:* player_database, current_gw, fixtures, offline_cache.
   - *Methods:* load_data(), get_player_stats(player_id), step_gameweek(), validate_squad_constraints(squad).
   - *Relationship:* Acts as the simulation world; validates squad rules and provides state updates to the agent.

2. **BaseFPLAgent (Abstract Base Class):**
   - *Attributes:* team_id, squad, bank, free_transfers.
   - *Methods:* evaluate_squad(), select_lineup() *(abstract)*, make_transfers() *(abstract)*.
   - *Subclasses:* GreedyBaselineAgent and AStarFPLAgent (implements custom priority-queue A* search and overrides select_lineup and make_transfers).

3. **PointPredictor:**
   - *Attributes:* model, feature_columns, is_trained.
   - *Methods:* engineer_features(df), train(train_data), predict_xp(test_data).
   - *Relationship:* Feeds estimated points into AStarFPLAgent to compute the heuristic and node costs.

4. **NotificationDispatcher:**
   - *Attributes:* telegram_token, chat_id.
   - *Methods:* format_weekly_payload(action_plan), send_alert().
   - *Relationship:* Takes the finalized actions from AStarFPLAgent and sends them to the user.

5. Scope

In scope — the features you commit to delivering. List the features that must work for the project to be considered complete.

#	Feature	Why it is essential
1	Automated REST Data Ingestion & Caching	Ingests real-time prices, injuries, and schedules, with local offline fallback for grading.
2	ML Expected Points Prediction Engine	Provides numeric point forecasts (xP) required by the search evaluation function.
3	Custom A* Squad & Transfer Search	Written from scratch to solve the combinatorial knapsack problem without using disallowed libraries.
4	Historical Season Backtester	Runs full 38-gameweek replays (2024/25) to measure performance against baselines with a fixed seed.
5	Automated Telegram Alert Dispatcher	Automatically notifies the user of starting XI, captain, and transfers before deadlines.

Out of scope — deliberately excluded. Naming what you will not build is as important as naming what you will; it shows that your scope is realistic for a group of two.

- **Automated live transfer execution on the FPL account:** Excluded to comply with Premier League Terms of Service and eliminate any risk of account bans.
- **In-match live substitutions and price-change day-trading:** Excluded because official FPL rules only allow team changes at fixed gameweek deadlines.
- **Computer vision / raw match video analysis:** Excluded because all player metrics are derived purely from official tabular match events.

6. Data or Environment Source

Source name and URL	Official Fantasy Premier League REST API (https://fantasy.premierleague.com/api/)
Type	Public dataset ☐ Web API ✓ Self-collected ☐ Generated / simulated ☐
Access requirements	No key needed ✓ Free key required ☐ (Paid sources are not permitted)
Approximate size	~15 MB per season (JSON fixtures, player history, summaries)
Licence / terms of use	Free public educational and personal research use. Strictly read-only access.
Offline fallback	The repository includes a data/raw/ directory containing complete, pre-downloaded JSON/CSV dumps of the 2024/25 season and completed rounds of 2026. If network calls fail or the project is graded offline, FPLEnvironment automatically loads these local snapshots.

7. Evaluation Plan

What will be compared	**Primary Technique:** Custom A* Search + ML (xP) vs. **Secondary Technique:** Local Beam Search + Rolling Form vs. **Baseline:** Greedy Form Heuristic Agent.
Metric(s)	Cumulative fantasy points across 38 gameweeks, search nodes expanded, runtime in seconds, prediction RMSE/MAE, and captaincy hit rate.

Baseline	Greedy heuristic agent that picks the highest historical point-scorers within budget with no lookahead.
Test instances / test set	Complete historical replay of the 2024/25 season (GW 1–38) and completed gameweeks of the 2026 season.
Number of runs and seed	5 runs per configuration with a fixed random seed (seed=42) for reproducibility.
Planned charts (at least three)	1. **Cumulative Points Trajectory Chart:** Weekly points progression (A* Agent vs. Beam Search vs. Greedy Baseline vs. Average Human Rank). 2. **Search Efficiency Plot:** Nodes expanded vs. Lookahead Horizon ($H \in \{1,2,3\}$) comparing A* against Beam Search. 3. **Prediction Error Plot:** Scatter plot and positional bar chart comparing Predicted xP vs. Realized Gameweek Points.

8. Division of Work

Member	Modules / components owned	Approximate share
Zumair Shamsi	Custom A* Search algorithm (search.py), node expansion, heuristic evaluation function, and Telegram alert module.	50%
Huzaifa Zafar	Data pipeline and offline caching (pipeline.py), Scikit-Learn xP model (models.py), and backtesting/charting suite.	50%

9. Milestones and Timeline

#	Milestone — what will be demonstrably working	Target date	Owner
1	Data pipeline downloads, parses, and loads offline JSON/CSV caches with zero network errors.	Week 6	Member 2
2	Baseline Greedy Agent and Scikit-Learn xP regressor run; outputs prediction RMSE on test gameweeks.	Week 8	Member 2
3	Custom A* search solves single-gameweek starting XI and captain optimization under valid constraints and prints nodes expanded.	Week 10	Leader
4	Multi-week lookahead transfer search and chip logic operational; backtester outputs comparison tables against baseline.	Week 12	Both
5	Experiments complete, final integration, documentation and repository cleanup	Week 14	Both

10. Expected Outcome and Risks

Expected Outcome (102 words):
When executed, the system ingests the manager's team ID, computes player point predictions, and executes the custom A* search in under 5 seconds. The console displays the Starting XI, Vice/Captain, recommended transfers, nodes expanded, and projected point gain, while a formatted alert is dispatched to Telegram. In historical season replays across 38 game weeks, we expect the A* agent to exceed 2,300 total points, outperforming the greedy baseline by at least 250 points and ranking in the top 5% of global players, while expanding over 70% fewer search nodes than unguided search.

Risks and Mitigation:

1. **API Rate Limiting / Offline Grading:** If the evaluator tests without internet, live requests will fail. *Mitigation:* The environment class implements an automatic offline fallback that loads pre-saved local JSON datasets from data/raw/ whenever network requests fail.

2. **Exponential State Space in Lookahead Search:** Expanding all 15-player transfer combinations over 3+ weeks could cause combinatorial explosion. *Mitigation:* We enforce an admissible heuristic to prune unviable branches and cap the search depth to a rolling 3-gameweek window with temporal discounting.

For office use — Lab Instructor	
Proposal status	Approved ☐ Approved with changes ☐ Revision required ☐
Comments	
Signature and date	

End of Project Proposal